

AI Engineering Portfolio Platform Agent Operating System Blueprint

A foolproof setup for using Claude Code as the primary coding agent, Codex as backup, and a safe in-repo AI agent layer with clean telemetry from day one.

Version: 1.0 - Scope: one-week MVP foundation plus safe optional mobile control path

The core principle: use coding agents to build the project, but make the project itself record enough structured evidence to show how the engineering work happened. The goal is not to automate everything immediately. The goal is clean data, low friction, and a system that can evolve without becoming expensive or fragile.

1. Executive Decision

Use two distinct agent types. Do not merge them too early.

Agent type	Purpose	Where it lives
Coding agents	Claude Code as main, Codex as backup. They edit files, run tests, plan work, and help implement features.	VS Code, terminal, local repo
In-repo project agent	A small backend-controlled agent/demo that can review code, manage todos, summarize work, and emit events.	Your application backend and database

This separation makes the setup safer. Claude Code and Codex can be powerful local builders, while the public-facing app-agent stays constrained, logged, rate-limited, and cheap.

2. Non-Negotiable Rules From Day One

- Every meaningful action becomes an event in the database or worklog.
- All AI API calls made by the application must go through one backend wrapper.
- No public user can run arbitrary expensive prompts in MVP.
- Claude Code is primary for coding tasks; Codex is backup and reviewer.
- No secrets in prompts, commits, screenshots, logs, or exported PDF documentation.
- No auth system in MVP unless absolutely necessary. Use constrained demos and server-side keys.
- Track token usage by model, category, feature area, task, and day.
- Keep local-first setup with docker compose so the project survives cloud/provider changes.

3. Target Repository Layout

```
ai-portfolio-platform/  
  frontend/           # Next.js + Tailwind app
```

```

backend/                # FastAPI app
  app/
    ai/                 # provider wrappers and agent orchestration
    api/               # routes
    db/               # models, migrations, repositories
    telemetry/        # event emitters, cost calculators, logging
    jobs/             # nightly/scheduled aggregation jobs
infra/                 # Terraform
scripts/              # local utilities and importers
docs/
  agent-os/           # this operating system and agent rules
  architecture/
  worklog/
.ai/
  PROJECT_CONTEXT.md
  AGENT_CONTRACT.md
  TELEMETRY_RULES.md
  TASK_TEMPLATE.md
  TOKEN_BUDGET.md
  MOBILE_CONTROL_POLICY.md
.github/
  workflows/
  copilot-instructions.md
AGENTS.md
CLAUDE.md
docker-compose.yml
.env.example
README.md

```

AGENTS.md gives generic coding-agent instructions. CLAUDE.md gives Claude-specific project memory. The .ai folder contains stricter internal contracts that both Claude and Codex must read before doing meaningful work.

4. Step-by-Step Setup: Coding Agents in VS Code

The goal is a consistent loop where Claude Code does most implementation work, Codex provides backup, second opinions, and cheaper scoped edits when appropriate.

- 1 Install VS Code and keep the repo opened at the workspace root.
- 2 Install Claude Code for VS Code and sign in. Use Claude Code as the main coding agent for planning and implementation tasks.
- 3 Install the Codex IDE extension or Codex CLI. Use it for backup implementation, review, bug isolation, and independent second opinions.
- 4 Create AGENTS.md, CLAUDE.md, and the .ai instruction folder before generating significant project code.
- 5 Create .env.example immediately, but never commit real API keys.
- 6 Create docs/worklog/WORKLOG.md and docs/decisions/DECISIONS.md before the first real feature.
- 7 Create the initial event schema before building the dashboard. Dirty early data is the biggest avoidable failure mode.
- 8 Start every coding-agent session by asking the agent to read the project context, current todos, telemetry rules, and token budget.
- 9 End every coding-agent session by asking the agent to summarize work, update docs, and list any telemetry gaps.

5. Agent Responsibility Split

Work type	Primary	Backup / reviewer
Architecture decisions	Claude Code	Codex review
Backend implementation	Claude Code	Codex focused diff review
Frontend implementation	Claude Code	Codex for component cleanup

Work type	Primary	Backup / reviewer
Terraform / deploy	Claude Code with manual review	Codex second opinion before apply
Bug hunting	Codex or Claude	The other agent reviews fix
Prompt/schema design	Claude Code	Codex checks edge cases
Cost/token review	Codex	Manual sanity check

6. Claude Code Session Prompt Template

Read these first:

- CLAUDE.md
- AGENTS.md
- .ai/PROJECT_CONTEXT.md
- .ai/AGENT_CONTRACT.md
- .ai/TELEMETRY_RULES.md
- .ai/TOKEN_BUDGET.md
- docs/worklog/WORKLOG.md
- docs/decisions/DECISIONS.md

Task:
[write the concrete task here]

Constraints:

- Keep MVP scope.
- Do not add auth, realtime, Kubernetes, vector DB, Redis, or new cloud services unless explicitly requested.
- All application AI calls must go through backend/app/ai/client.py.
- Every meaningful app action must emit an event.
- Update todos/worklog/decisions when relevant.
- Prefer small, reviewable diffs.
- Run tests or explain why they were not run.
- End with: changed files, tests run, telemetry impact, token/cost risk.

7. Codex Backup Prompt Template

Act as a backup coding reviewer. First read AGENTS.md, .ai/PROJECT_CONTEXT.md, and .ai/TELEMETRY_RULES.md.

Review this change/task for:

1. correctness
2. overengineering
3. telemetry gaps
4. cost/security risks
5. missing tests

Do not rewrite everything. Return a concise patch plan and only change files if asked.

8. Instruction Files: Minimum Contents

These files are the project control layer. They make agent output more consistent and make the development process easier to audit later.

File	Purpose	Must include
AGENTS.md	General coding-agent onboarding	Setup commands, tests, repo map, coding style, no-go scope
CLAUDE.md	Claude Code project memory	MVP goal, architecture, agent rules, end-of-session checklist
.ai/PROJECT_CONTEXT.md	Stable project summary	Product goal, target users, stack, MVP boundaries
.ai/AGENT_CONTRACT.md	Rules for all AI-assisted coding	Allowed actions, forbidden actions, required documentation updates
.ai/TELEMETRY_RULES.md	Logging contract	Event types, required fields, cost fields, feature_area taxonomy

File	Purpose	Must include
.ai/TOKEN_BUDGET.md	Cost control	Daily budgets, escalation triggers, model usage policy
.ai/TASK_TEMPLATE.md	Repeatable task format	Goal, scope, files, acceptance criteria, telemetry impact
.ai/MOBILE_CONTROL_POLICY.md	Optional mobile safety rules	Allowed commands, denylist, approval rules

9. Suggested AGENTS.md Skeleton

```
# AGENTS.md

## Project goal
Build a cloud-based AI engineering portfolio platform with clean telemetry, low cost, and a polished MVP.

## Setup commands
- Backend: cd backend && uv sync && uv run pytest
- Frontend: cd frontend && npm install && npm run dev
- Full stack: docker compose up

## Architecture rules
- AI providers are called only through backend/app/ai/client.py.
- Meaningful actions must emit events through backend/app/telemetry/events.py.
- Do not add services unless required by the task.
- Prefer Postgres JSONB over adding new infrastructure.

## Scope restrictions
No auth, Kubernetes, realtime, vector DB, Redis, payments, or multi-user SaaS behavior in MVP.

## End-of-task checklist
- Run relevant tests.
- Update WORKLOG.md.
- Update DECISIONS.md if architecture changed.
- Note telemetry impact.
- Note cost/token risk.
```

10. Suggested CLAUDE.md Skeleton

```
# CLAUDE.md

You are the primary coding agent for this repository.

Core priorities:
1. Keep the MVP simple and deployable.
2. Protect data quality for telemetry from day one.
3. Avoid overengineering.
4. Keep changes small and reviewable.
5. Prefer established, low-cost services.

Before major work:
- Read .ai/PROJECT_CONTEXT.md
- Read .ai/AGENT_CONTRACT.md
- Read .ai/TELEMETRY_RULES.md
- Read .ai/TOKEN_BUDGET.md

After major work:
- Update docs/worklog/WORKLOG.md
- Update docs/decisions/DECISIONS.md if needed
- Summarize changed files, tests, telemetry, and risks
```

11. Telemetry Schema: Get This Right Early

The telemetry system should not depend on manually written perfect notes. Use a hybrid approach: automatic logging for application events, and lightweight manual/session logging for coding-agent usage.

Core event table

```
CREATE TABLE events (
```

```

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
event_type TEXT NOT NULL,
source TEXT NOT NULL,          -- app, github, manual, scheduled_job, mobile
feature_area TEXT,            -- frontend, backend, infra, ai, docs, observability, todo_agent
task_id UUID,
commit_sha TEXT,
model_provider TEXT,
model_name TEXT,
input_tokens INTEGER DEFAULT 0,
output_tokens INTEGER DEFAULT 0,
total_tokens INTEGER DEFAULT 0,
estimated_cost_usd NUMERIC(12,6) DEFAULT 0,
latency_ms INTEGER,
success BOOLEAN,
metadata JSONB NOT NULL DEFAULT '{}'::jsonb
);

CREATE INDEX idx_events_created_at ON events(created_at);
CREATE INDEX idx_events_type ON events(event_type);
CREATE INDEX idx_events_feature_area ON events(feature_area);
CREATE INDEX idx_events_model ON events(model_provider, model_name);

```

Required event types

Event type	Source	Why it matters
ai_request_completed	app/backend	Token and cost accounting
coding_session_logged	manual/CLI	Tracks Claude/Codex usage while building
todo_created	app/manual	Project management timeline
todo_completed	app/manual/GitHub	Links progress to commits
commit_pushed	GitHub Action	Engineering activity
deploy_completed	CI/CD	Deployment frequency
daily_stats_aggregated	scheduled_job	Dashboard performance
error_recorded	app/backend	Reliability evidence

12. Coding-Agent Usage Logging

Claude Code and Codex usage data may not be perfectly available through APIs or extensions. Do not block the project on full automation. Start with structured session logging, then automate later if the tools expose stable data.

Manual/semi-automatic session logger

```
python scripts/log_ai_session.py
```

Prompted fields:

- provider: anthropic | openai | other
- model: claude-sonnet | claude-opus | gpt-codex | other
- session_type: coding | debugging | architecture | writing | review | other
- feature_area: frontend | backend | infra | ai | docs | observability | todo_agent
- task_id: optional
- prompt_count: integer
- input_tokens: optional integer
- output_tokens: optional integer
- cost_estimate_usd: optional decimal
- summary: short text
- changed_files: optional list

This is not glamorous, but it protects data quality. One 30-second log after each meaningful session is enough for the MVP.

Daily token view

Daily dashboard cards:

- tokens_today_total
- tokens_today_by_provider
- tokens_today_by_model
- tokens_today_by_feature_area
- prompts_today
- estimated_cost_today_usd
- highest_cost_task_today
- 7_day_token_trend

13. AI Wrapper: Application Agent Calls

The in-repo app-agent must never call Claude/OpenAI directly from random routes. Every application AI call should pass through one wrapper that logs metadata.

```
# backend/app/ai/client.py

class AIClient:
    def generate(self, *, task_type, feature_area, prompt, model=None, metadata=None):
        start = now()
        try:
            response = provider.generate(prompt=prompt, model=model)
            usage = extract_usage(response)
            emit_event(
                event_type='ai_request_completed',
                source='app',
                feature_area=feature_area,
                model_provider=usage.provider,
                model_name=usage.model,
                input_tokens=usage.input_tokens,
                output_tokens=usage.output_tokens,
                total_tokens=usage.total_tokens,
                estimated_cost_usd=estimate_cost(usage),
                latency_ms=elapsed_ms(start),
                success=True,
                metadata={'task_type': task_type, **(metadata or {})}
            )
            return response
        except Exception as exc:
            emit_event(event_type='error_recorded', source='app', success=False,
                      feature_area=feature_area, metadata={'error': str(exc)})
            raise
```

14. Token Budget Policy

Claude can be extremely useful, but long agent sessions can burn tokens quickly. Use policy, not vibes.

Use case	Preferred model/tool	Budget rule
Major architecture/design	Claude Code	Accept higher token use, but require summary and decision log
Implementation of scoped task	Claude Code	One focused session per task where possible
Diff review	Codex	Keep cheap and bounded
Bug isolation	Codex or Claude	Use smallest context that reproduces issue
Docs cleanup	Codex or cheaper model	Avoid premium model unless text is strategic
App demo agent	Cheap model by default	Hard input limits and rate limits

Escalation triggers

- If daily AI cost crosses the soft budget, stop broad exploratory prompting and use scoped tasks only.
- If one task uses unusually high tokens, log why in WORKLOG.md.
- If the agent repeatedly fails, switch model or manually implement rather than burning more tokens.
- If a context window becomes huge, ask the agent to create a compact handoff summary and start fresh.

15. Todo Co-Pilot Setup

The todo co-pilot should reduce overhead, not become a complicated product. It should operate on structured tasks and events.

```
CREATE TABLE todos (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
  completed_at TIMESTAMPTZ,  
  title TEXT NOT NULL,  
  description TEXT,  
  status TEXT NOT NULL DEFAULT 'open',  
  priority TEXT NOT NULL DEFAULT 'medium',  
  feature_area TEXT NOT NULL,  
  linked_commit_sha TEXT,  
  ai_tokens_spent INTEGER DEFAULT 0,  
  prompts_count INTEGER DEFAULT 0,  
  metadata JSONB NOT NULL DEFAULT '{}'::jsonb  
);
```

Todo co-pilot MVP behavior

- Create tasks from a short natural-language instruction.
- Suggest 3-6 subtasks with feature_area labels.
- Never execute code or cloud commands from public/mobile input in MVP.
- Link completed tasks to commits manually or via GitHub Actions later.
- Show tokens/prompts spent per task and per feature area.

16. Safe Optional Mobile Control Path

Mobile prompting can be fun and useful, especially if you want to experiment with Swift. Keep it rational: the phone should create or approve tasks, not freely mutate production infrastructure.

Recommended mobile phases

Phase	Mobile ability	Safety level
M0	View dashboard, todos, daily tokens	Safe read-only
M1	Create todo items and notes	Safe write to constrained tables
M2	Ask project agent for task breakdowns	Constrained AI call with rate limits
M3	Approve a pre-generated GitHub issue/task	Requires explicit confirmation
M4	Trigger safe GitHub Action workflow	Only allow pre-approved workflows
Never in MVP	Run shell commands, terraform apply, arbitrary repo edits from phone	Too risky

Mobile safety contract

```
Mobile app can:  
- read dashboard stats  
- create todos  
- add notes  
- request summaries  
- trigger constrained demo-agent runs  
  
Mobile app cannot:  
- access raw provider API keys  
- run shell commands  
- run terraform apply  
- deploy to production directly  
- edit arbitrary files
```

- execute arbitrary prompts against large repo context
- bypass rate limits or logging

All mobile actions must emit events with source='mobile'.

Swift experiment recommendation

A small Swift app is a good optional learning project, but do not build it before the web dashboard and backend event system work. The Swift app should call your backend, not Anthropic/OpenAI directly. That keeps keys server-side and makes mobile actions visible in the same telemetry pipeline.

17. One-Week Foundation Plan

Day	Outcome	Acceptance criteria
0	Repo control files	AGENTS.md, CLAUDE.md, .ai files, README skeleton exist
1	Local-first skeleton	docker compose starts frontend/backend/db
2	Event schema	events table exists, emit_event works, one test passes
3	Coding session logger	scripts/log_ai_session.py writes coding_session_logged event
4	AI wrapper	App AI call logs tokens/cost/latency through wrapper
5	Todo tables/API	Create/list/update todos, emit todo events
6	Dashboard v1	Daily tokens by model, prompts today, todos, recent events
7	Docs and demo polish	README, architecture diagram, sample data, deployment notes

18. Foolproof Work Session Loop

- 1 Pick one todo with a clear feature_area.
- 2 Open VS Code at repo root.
- 3 Ask Claude Code to read the required context files.
- 4 Ask for a plan before code if the task touches architecture, database, infra, or AI wrapper.
- 5 Let Claude implement a small diff.
- 6 Run tests locally.
- 7 Use Codex to review the diff if the change is important.
- 8 Commit with a clear message and linked todo id if available.
- 9 Run scripts/log_ai_session.py and record provider/model/tokens/category/feature_area.
- 10 Update WORKLOG.md and DECISIONS.md if needed.

19. Data Quality Guardrails

The dashboard will only be impressive if the early data is consistent. Use controlled vocabularies and avoid free-text chaos where structured fields are possible.

Controlled vocabularies

```
feature_area:
- frontend
- backend
- infra
- ai
- observability
- docs
```



```

- todo_agent
- mobile

session_type:
- coding
- debugging
- architecture
- writing
- review
- planning
- other

model_provider:
- anthropic
- openai
- local
- other

source:
- app
- manual
- github
- scheduled_job
- mobile

```

Minimum viable analytics queries

```

-- Daily tokens by model
SELECT date_trunc('day', created_at) AS day, model_provider, model_name,
       SUM(total_tokens) AS tokens
FROM events
WHERE total_tokens > 0
GROUP BY 1,2,3
ORDER BY day DESC;

-- Tokens by feature area
SELECT feature_area, SUM(total_tokens) AS tokens
FROM events
GROUP BY feature_area
ORDER BY tokens DESC;

-- Prompts today
SELECT COUNT(*)
FROM events
WHERE event_type IN ('ai_request_completed', 'coding_session_logged')
      AND created_at::date = CURRENT_DATE;

```

20. What To Put On The Public Website

- Live system status: frontend, backend, database, latest deploy.
- Daily tokens by model and provider.
- Cumulative prompts and estimated cost.
- Feature-area breakdown: frontend, backend, infra, AI, docs, observability.
- Todo progress and completed tasks.
- Recent deploys and commits.
- Architecture diagram and explanation of design tradeoffs.
- Controlled public AI demo with limited input and clear cost guardrails.

21. Hiring Signal Summary

Signal	How this setup proves it
AI engineering	Provider wrapper, structured prompts, telemetry, cost tracking
Full-stack	Next.js dashboard plus FastAPI backend

Signal	How this setup proves it
Cloud readiness	Deployable app, Terraform, CI/CD, environment handling
Data science thinking	Event logging, daily aggregation, trend dashboards
Operational maturity	Cost controls, rate limits, safety boundaries
Agentic workflow competence	Claude/Codex repo contracts plus app-agent demo
Documentation quality	AGENTS.md, CLAUDE.md, README, worklog, decisions

22. Final Recommendation

Build the agent operating system before building fancy features. The first few days should produce the repo contracts, event schema, session logger, AI wrapper, and dashboard foundation. That makes the rest of the project safer, cheaper, and more impressive because every later feature automatically creates evidence of how it was built.

References

Anthropic. Claude Code product and VS Code documentation. Accessed May 22, 2026. <https://claude.com/product/claude-code> and <https://code.claude.com/docs/en/vs-code>

Anthropic Engineering. Claude Code auto mode safety discussion. Accessed May 22, 2026. <https://www.anthropic.com/engineering/claude-code-auto-mode>

OpenAI. Codex IDE extension documentation. Accessed May 22, 2026. <https://developers.openai.com/codex/ide>

OpenAI. Codex IDE extension features. Accessed May 22, 2026. <https://developers.openai.com/codex/ide/features>

AGENTS.md project. Agent instruction file format. Accessed May 22, 2026. <https://agents.md/>